

# Contents

## Module 1: Lecture Script (3 hours, TR + EN)

1

## Module 1: Lecture Script (3 hours, TR + EN)

Speaker notes for presenting `module-1-introduction.pdf` live. Not part of the published site (instructor-only prep material) - excluded from the mkdocs build the same way `reports/README.md` is.

This is a *speaking script*, not a word-for-word transcript to read aloud. Say the TR or EN block, not both; they cover the same content, use whichever language fits the room that day. On the discussion-heavy slides, pause and actually take the questions before moving on rather than answering them yourself.

### Pacing (180 minutes total, including one break)

| #  | Slide                              | Minutes | Cumulative |
|----|------------------------------------|---------|------------|
| 1  | Cover                              | 5       | 5          |
| 2  | What this course covers            | 10      | 15         |
| 3  | The roadmap                        | 10      | 25         |
| 4  | Where to find things (demo)        | 8       | 33         |
| 5  | Learning objectives                | 5       | 38         |
| 6  | Architecture versus design         | 15      | 53         |
| 7  | Why it matters: the cost of change | 15      | 68         |
| -  | <b>Break</b>                       | 10      | 78         |
| 8  | Quality attributes                 | 20      | 98         |
| 9  | A trade-off, made concrete         | 10      | 108        |
| 10 | Stakeholders rarely agree          | 15      | 123        |
| 11 | Project: quality attribute audit   | 12      | 135        |
| 12 | Choosing a project domain          | 12      | 147        |
| 13 | Summary                            | 8       | 155        |
| 14 | Next week                          | 5       | 160        |
| -  | Questions and wrap-up              | 20      | 180        |

### 1. Cover - 5 min

**TR:** Herkese hoş geldiniz. Bu, Software Design and Architecture dersinin ilk oturumu. Ben [isim], bu dönem boyunca sizinle birlikte olacağım. Bu ders on iki hafta sürüyor: on modül ve iki proje sprinti. Bugün ilk modülle başlıyoruz: mimari nedir, tasarımdan nasıl ayrılır, ve bu ayrımın neden gerçekten önemli olduğu.

**EN:** Welcome, everyone. This is the first session of Software Design and Architecture. I'm [name], and I'll be with you for the whole semester. The course runs twelve weeks: ten modules plus two project sprints. Today we start with the first module: what architecture is, how it differs from design, and why that distinction actually matters.

### 2. What this course covers - 10 min

**TR:** Bu ders tek bir sınıfın içindeki prensiplerden, dağıtık bir sistemin bütün trade-off'larına kadar ilerliyor, ve bu kararları başkasının güvенеbileceği şekilde nasıl yazıya döktüğümüzü de öğreniyor. On modül, iki proje sprinti, ve bir grup projesi: dönem boyunca büyüyen, tek bir proje. Sprint A'dan Sprint B'ye kadar

aynı proje üzerinde çalışacaksınız, atmayacaksınız. [DURAKLA: “Şimdiden bir proje fikriniz var mı?” diye sorup birkaç yanıt alın.]

**EN:** This course moves from the principles inside one class to the trade-offs of a whole distributed system, and how we write those decisions down so someone else can trust them. Ten modules, two project sprints, and one group project: a single project that grows through the whole term. You’ll work on the same project from Sprint A to Sprint B, not throw it away. [PAUSE: ask “Does anyone already have a project idea?” and take a couple of answers.]

### 3. The roadmap - 10 min

**TR:** Bu, önümüzdeki on iki haftanın tam haritası. İlk beş hafta temel: bugün mimari ve tasarım ayrımı, sonra nesne yönelimli tasarım prensipleri, tasarım kalıpları, ve mimari stiller. Altıncı hafta Sprint A: yeni içerik yok, sadece o ana kadar öğrendiklerinizi kendi projenize uyguluyorsunuz. Yedinci haftadan on birinci haftaya kadar ölçekte sistemler geliyor: domain-driven design, mikroservisler, dağıtık sistemler ve dayanıklılık, mimari dokümantasyon, ve kalite değerlendirmesi. On ikinci hafta Sprint B, final teslim haftası.

**EN:** This is the full map of the next twelve weeks. The first five weeks are the foundation: today’s architecture-versus-design distinction, then object-oriented design principles, design patterns, and architectural styles. Week six is Sprint A: no new lecture content, you apply everything so far to your own project. Weeks seven through eleven cover systems at scale: domain-driven design, microservices, distributed systems and resilience, architecture documentation, and quality evaluation. Week twelve is Sprint B, the final submission week.

### 4. Where to find things - 8 min

**TR:** Nerede ne bulacağınızı bir kere söyleyeyim, tekrarlamayacağım. Her şey tek bir GitHub reposunda. Bu haftanın modülü `m1_introduction/README.md` dosyasında. Her modülün kendi `exercise_files/` klasörü var. Rapor şablonu `reports/README.md`’de, ve şimdiden orada, sadece dönem sonunda değil. [CANLI GÖSTERİM: repoyu ve GitHub Pages sitesini açın, sol menüden bir modüle tıklayın, “Slides” sayfasını gösterin.]

**EN:** Let me tell you where everything lives, once, so I don’t have to repeat it. Everything is in one GitHub repository. This week’s module is `m1_introduction/README.md`. Every module has its own `exercise_files/` folder. The report template lives in `reports/README.md`, and it’s there from week one, not just at the end. [LIVE DEMO: open the repo and the GitHub Pages site, click into a module from the left-hand nav, show the “Slides” page.]

### 5. Learning objectives - 5 min

**TR:** Bugünün sonunda dördünüzü yapabilmelisiniz: mimariyi tasarımdan ayırmak ve bu ayrımın neden önemli olduğunu bilmek, teknik borcu bir metafor olarak anlamak, temel kalite niteliklerini tanımak ve aralarındaki trade-off’ları adlandırabilmek, ve bir mimariyi şekillendiren stakeholder’ları tanımlamak. Bunların hiçbirisi kod yazmak değil, ama bundan sonraki her modül bu kelime dağarcığının üzerine kuruluyor.

**EN:** By the end of today you should be able to do four things: distinguish architecture from design and know why the distinction matters, understand technical debt as a metaphor, recognize the major quality attributes and name the trade-offs between them, and identify the stakeholders who shape an architecture. None of this is writing code, but every module after this one builds on this vocabulary.

### 6. Architecture versus design - 15 min

**TR:** İşte dersin tam kalbi. Tasarım, bir bileşenin içindeki kararlar: bir sınıfı düzenlemek, genelde bir öğleden sonrada değiştirebileceğiniz bir şey. Mimari, sistemin yapısıyla ilgili kararlar: değiştirmek için gerçek bir göç planına ihtiyacınız olur. Sıralama algoritmasını değiştirmek tasarımdır. İlişkisel bir veritabanını, her servisin

okuduğu bir event log ile değiştirmek mimaridir. [TARTIŞMA: sınıftan kendi örneklerini isteyin, bir tasarım kararı ve bir mimari karar; ikisi arasındaki farkı birlikte bulun.]

**EN:** Here's the real heart of the course. Design is decisions inside one component: editing a class, usually something you can change in an afternoon. Architecture is decisions about the system's structure: changing it needs a real migration plan. Swapping a sorting algorithm is design. Swapping a relational database for an event log that every service reads from is architecture. [DISCUSSION: ask the class for their own examples, one design decision and one architectural decision; work out the difference together.]

## 7. Why it matters: the cost of change - 15 min

**TR:** Teknik borç, finansal borca benzer: bir kısayol sizi şimdi hızlandırır, ama aynı alana dokunan her gelecekteki değişiklikte faiz öder. Fark şu: finansal borcun bilinen bir faiz oranı ve vade tarihi var; teknik borcun "faiz oranı" genelde görünmez, ta ki bir ekip ilgisiz bir değişiklik yapmaya çalışıp kodun ne kadar karışık olduğunu keşfedene kadar. Maliyet eğrisi basit: aynı yapısal hata, ne kadar geç yakalanırsa o kadar pahalı olur, kabaca on kat her aşamada. [CANLI GÖSTERİM değil, TARTIŞMA: birinin production'da yakaladığı bir hatayı, daha erken yakalanmış olsaydı ne kadar ucuz olacağını hayal edin.]

**EN:** Technical debt resembles financial debt: a shortcut moves you faster now, at the cost of interest paid on every future change that touches the same area. The difference: financial debt has a known interest rate and a payoff date; technical debt's "interest rate" is usually invisible until a team tries to make an unrelated change and discovers how tangled the code has become. The cost curve is simple: the same structural mistake costs roughly ten times more the later it's caught. [Not a live demo, a DISCUSSION: imagine a bug someone caught in production, and how much cheaper it would have been caught earlier.]

## Break - 10 min

**TR:** On dakika ara verelim. Geri geldiğimizde kalite niteliklerine, trade-off'lara, ve stakeholder'lara geçiyoruz.

**EN:** Let's take a ten-minute break. When we're back, we move to quality attributes, trade-offs, and stakeholders.

## 8. Quality attributes - 20 min

**TR:** Bu tablodaki yedi nitelik, "-ility" ile bitenler: performans, kullanılabilirlik, güvenlik, sürdürülebilirlik, test edilebilirlik, ölçeklenebilirlik, kullanılabilirlik. Hiçbir mimari hepsini aynı anda optimize etmez, çünkü birini iyileştirmek genelde bir başkasına mal olur. Her satırı okuyalım ve her birinin gerçek bir maliyeti olduğunu vurgulayalım; bu tablo ezberlenecek bir liste değil, bundan sonraki her modülün üzerine kurulacağı bir çerçeve. [TARTIŞMA: "Sizce kendi telefonunuzdaki bir uygulama hangi iki niteliği en çok önceliklendiriyor?"]

**EN:** These seven attributes in the table, the ones ending in "-ility": performance, availability, security, maintainability, testability, scalability, usability. No architecture optimizes all of them at once, since improving one usually costs another. Let's read through each row and stress that every one has a real cost; this table isn't a list to memorize, it's a framework every later module builds on. [DISCUSSION: "Which two attributes do you think an app on your own phone prioritizes most?"]

## 9. A trade-off, made concrete - 10 min

**TR:** Soyut kalsın istemiyorum, o yüzden tek bir somut örnek: bir cache eklemek performansı iyileştirir, ama sürdürülebilirliğe mal olur, çünkü artık aynı verinin iki kopyası birbiriyle çelişebilir ve her gelecekteki değişiklik ikisini de düşünmek zorunda kalır. Bu, önceki slaytın tablosunun neden sadece bir liste olmadığını gösteren tek bir örnek.

**EN:** I don't want this to stay abstract, so here's one concrete example: adding a cache improves performance, at the cost of maintainability, since now two copies of the same data can disagree and every future

change has to reason about both. This is the one example that shows why the previous slide's table is more than just a list.

## 10. Stakeholders rarely agree - 15 min

**TR:** Bir mimarinin tek bir izleyicisi yok. Ürün sahibi zamana ve özelliklere odaklanır. Operasyon ekibi kullanılabilirlik ve gözlemlenebilirliğe. Güvenlik ekibi saldırı yüzeyine. Ve gelecekteki mühendis, yani altı ay sonraki siz, sürdürülebilirliğe. Bir demoya en hızlı yol genelde sürdürülebilirliğe zarar verir; en güvenli tasarım genelde kullanılabilirliğe zarar verir. [TARTIŞMA: dört kartı gösterip her birinin sınıftan bir öğrenciye “role” verin, o rolün bakış açısından tek bir cümle söylemelerini isteyin.]

**EN:** An architecture rarely has one audience. A product owner cares about time-to-market and features. An operations team cares about availability and observability. A security team cares about attack surface. And the future engineer, quite possibly you in six months, cares about maintainability. The fastest path to a demo often damages maintainability; the most secure design often damages usability. [DISCUSSION: show the four cards and assign each to a student as a role, have them say one sentence from that role's point of view.]

## 11. Project: quality attribute audit - 12 min

**TR:** Bugünün zorunlu projesi: küçük, gerçek bir açık kaynak projesi seçin, dizin ve modül yapısını okuyun, sonra üç şey yapın: yapının hangi iki-üç kalite niteliğini önceliklendirdiğini gerçek kanıtla belirleyin, hangi bir niteliği feda etmiş gibi görüldüğünü kanıtla belirleyin, ve kanıtı sonucunuza bağlayan 200-300 kelime yazın. “Sürdürülebilir görünüyor” kanıt değildir; “her dış API çağırısı tek bir `Client` arayüzünün arkasına gizlenmiş” kanıttır. [DURAKLA: “Herkes bir proje seçti mi, yardıma ihtiyacı olan var mı?”]

**EN:** Today's required project: pick a small, real open-source project, read its directory and module structure, then do three things: identify which two or three quality attributes the structure seems to prioritize, with real evidence; identify one attribute it seems to have sacrificed, with evidence; and write 200-300 words connecting the evidence to your conclusion. “It looks maintainable” is not evidence; “every external API call is hidden behind one `Client` interface” is. [PAUSE: “Has everyone picked a project, does anyone need help?”]

## 12. Choosing a project domain - 12 min

**TR:** Şimdi bu dönemin grup projesine geçelim. Yeterince gerçek karmaşıklığı olan bir şey seçin: bir domain modeline, en az iki doğal servis sınırına, ve üzerine bir ADR yazmaya değer bir karara ihtiyaç duysun. Slaytta dokuz başlangıç noktası var, kütüphane yönetiminden çok oyunculu bir oyun sunucusuna kadar. Bunlar sabit bir menü değil, kendi alanınızı getirmek isterseniz, gerçek bir yapısı olduğu sürece sorun yok. [TARTIŞMA: gruplar oluşsun ve 5 dakika içinde bir domain üzerinde anlaşılmaya çalışsınlar.]

**EN:** Now let's move to this term's group project. Pick something with enough real complexity: it should need a domain model, at least two natural service boundaries, and a decision worth writing an ADR about. The slide lists nine starting points, from a library management system to a multiplayer game backend. These aren't a fixed menu; bring your own domain if you want, as long as it has real structure to it. [DISCUSSION: let groups form and try to agree on a domain within 5 minutes.]

## 13. Summary - 8 min

**TR:** İkinci modül başlamadan önce şunları biliyor olmalısınız: mimari ve tasarım, bir kararın geri alınmasının ne kadar pahalı olduğuyla ayrılır; teknik borç, yapısal kararlar ucuzken yapılabilecekleri noktayı geçtiğinde birikir; her mimari karar bir kalite niteliğini bir diğerine karşı takas eder; ve stakeholder'lar hangi niteliğin en önemli olduğu konusunda nadiren anlaşır, bu yüzden bunu açıkça yazmak gerekir, varsaymak değil.

**EN:** Before Module 2 starts, you should know: architecture and design differ mainly in how expensive a decision is to reverse; technical debt accumulates when structural decisions get deferred past the point where they were cheap to make well; every architectural decision trades one quality attribute against another; and stakeholders rarely agree on which attribute matters most, which is exactly why it has to be written down explicitly, not assumed.

#### **14. Next week - 5 min**

**TR:** Gelecek hafta ikinci modül: nesne yönelimli tasarım prensipleri. Coupling, cohesion, ve beş SOLID prensibi, her biri gerçek bir önce-sonra refactor örneğiyle. Bugünkü zorunlu projeyi ikinci modülden önce bitirin.

**EN:** Next week is Module 2: object-oriented design principles. Coupling, cohesion, and the five SOLID principles, each with a real before-and-after refactor example. Finish today's required project before Module 2.

#### **15. Questions and wrap-up - 20 min**

**TR:** Sorularınızı alalım. [DURAKLA, gerçekten bekleyin, sessizlik rahatsız edici gelse bile.] Kimse soru sormazsa, kendiniz bir soru sorun: "Kendi projenizde bugün gördüğümüz bir trade-off'u şimdiden görebiliyor musunuz?" Bu, herkesi ilk projeye bağlamanın iyi bir yolu.

**EN:** Let's take questions. [PAUSE, actually wait, even if the silence feels uncomfortable.] If nobody asks anything, ask one yourself: "Can you already spot one of today's trade-offs in your own project?" That's a good way to connect everyone back to the first project.